A portable `std::simd` FFT, up to 3.4× faster than GNU Radio 4's

Dennis Klein Claude (Anthropic)

October 2026

Abstract

GNU Radio 4 computes FFTs with `SimdFFT`, a `std::simd` port of PFFFT whose vectors are fixed at four lanes. We present `fastfft`, a single-header radix-8/16 Stockham FFT written against `native_simd` without ISA intrinsics. On one AVX-512 core it is 1.6–3.4× faster than `SimdFFT` for complex single-precision transforms of size 2^6–2^{20} and 1.3–2.0× in double precision, and between 0.79–2.04× (float) and 0.70–1.21× (double) as fast as FFTW 3.3.10 with AVX-512 codelets. Inside GNU Radio 4, a small patch makes the streaming FFT block 1.7–2.5× faster (float). Using the full vector width accounts for most of the gain. Writing the kernel against `std::simd` required three workarounds, which we describe. The code and a harness that repeats all measurements on any Linux host are public.

1 Introduction

GNU Radio 4 (gr4) [1] is a header-only C++23 rewrite of the GNU Radio signal-processing framework. Its FFT blocks call `gr::algorithm::FFT`, which dispatches to `SimdFFT`, a port of PFFFT [2] to `std::experimental::simd` [3] via vir-simd [4]. `SimdFFT` uses `simd<T, 4>`: four floats or four doubles, whatever the hardware offers. On an AVX-512 machine this leaves three quarters of each register unused in single precision.

We asked how fast a power-of-two complex FFT can be when written only against `native_simd`, so that the width follows the compilation target, and whether it can match a tuned library. The answer, `fastfft`, is one 400 line header. This note describes its structure (§2), what `std::simd` could not express (§3), the measurement method (§4) and the results (§5).

2 Algorithm

`fastfft` is a Stockham autosort FFT [6, 7]: every pass reads one buffer and writes the other in natural order, so no bit-reversal pass is needed. Data are interleaved complex numbers; a vector of W scalars holds $C = W/2$ complex values (8 floats or 4 doubles per 512 bit register).

1. *First pass, radix 8, vectorised over the butterfly index.* With stride 1 the eight inputs of a butterfly are $N/8$ apart, so C consecutive butterflies form one vector per input. The eight results of each butterfly must be stored contiguously; an in-register $8 \times C$ complex transpose (three rounds of two-input zips) does this before the store.
2. *Middle passes, radix 16, vectorised over the stride.* Once the stride $s \geq C$, the C lanes of a vector are C independent sub-transforms that share a twiddle factor, which is broadcast. A radix-16 butterfly is done as 4×4 in registers. Each pass reads and writes the data once, so a 2^{20} transform takes six passes instead of ten for radix 4.
3. *Last pass,* radix 2, 4 or 8, whatever remains.

Complex multiplication by a twiddle w uses tables stored as (w_r, w_r) and $(-w_i, w_i)$ per lane pair, so $z \cdot w = z \odot \mathbf{w_r} + \mathrm{swap}(z) \odot \mathbf{w_i'}$ costs one multiply, one fused multiply-add and one in-lane swap. Twiddles are precomputed in double precision. Sizes below 64 use a scalar radix-2 path.

3 What `std::simd` could not express

The kernel uses `std::experimental::simd` as shipped with libstdc++ and contains no intrinsics, but three points needed care. **(i) Two-input shuffles.** The transpose in pass 1 needs zip operations that take lanes from two vectors. `std::simd` has none; `concat` followed by a permute compiled to element-wise moves that cost about three times the whole pass. We use the compiler's generic vector extension (`__builtin_shufflevector`), which GCC and clang lower to permute instructions (`vpermt2ps` on AVX-512). The re/im swap is written the same way. **(ii) FMA.** libstdc++'s `fma(simd,simd,simd)` is not inlined; we write `a*b+c` and compile with `-ffp-contract=fast` (GCC's default in GNU mode). **(iii) Backends.** vir-simd forwards to libstdc++'s implementation when it is available; with libc++ it falls back to emulating the vector types with arrays, silently (§5). Our header defines `FASTFFT_NATIVE_SIMD` so that callers can detect this case. Finally, with 128 bit registers a vector holds only one complex double; the header then uses a pair of registers (`simd<double,4>`).

4 Method

`fftbench` times `fastfft`, `SimdFFT` (gr4 rev. `af62235`) and FFTW 3.3.10 [5] (built from source with the host's SIMD codelets, `FFTW_MEASURE`) on the same input, uniform random in $[-1, 1)^2$. A *sample* transforms $\max(N, 2^{16})$ consecutive values as frames of N, the way a streaming gr4 block sees them, repeated until it lasts at least 5 ms; one warm-up sample is discarded. Before timing, every output is checked against a long-double reference DFT. Each process runs 40 rounds; in every round each implementation is sampled once in a freshly randomised order, so drifts affect all of them alike and same-round samples can be paired. The process is run three times and pinned to one core. We report the median of the 120 samples per cell with a 95 % percentile-bootstrap confidence interval, and speedups as the median of paired per-round ratios, also with a bootstrap CI [8, 9]. All values, raw samples and host data are in the repository; one script (`bench/run.sh`) repeats the whole procedure.

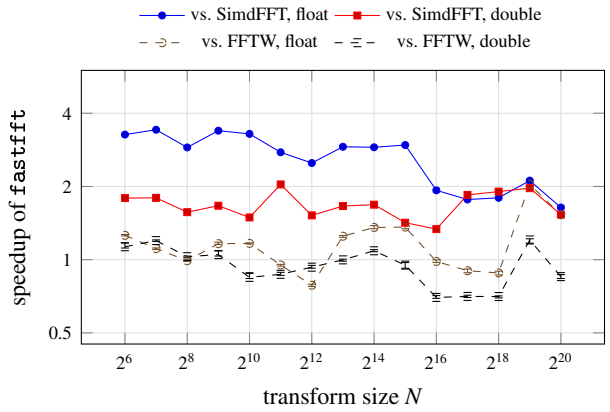


Figure 1: Speedup of `fastfft` (median of paired ratios, 95 % CI as error bars; mostly smaller than the markers). Above 1 means `fastfft` is faster. Absolute times in Tables 1 and 2.

Host: virtual machine with 4 vCPU of an Intel Xeon at 2.1 GHz nominal (AVX-512 incl. FP16, 2 MiB L2 per core), Linux 6.18.44-fc-v114, g++-14 14.2.0, `-O3 -march=native`. The largest relative median absolute deviation over all cells was 7.7 %.

5 Results

Kernel. Figure 1 summarises the comparison. In single precision `fastfft` is 2.5–3.4× faster than `SimdFFT` up to 2^{15} and 1.6–2.1× above, where both are increasingly bound by memory traffic. Against FFTW the picture depends on N (0.79–2.04×): `fastfft` is ahead at 2^6–2^7, 2^9–2^{10}, 2^{13}–2^{15} and 2^{19}–2^{20}, within 5 % at 2^8, 2^{11} and 2^{16}, and behind at 2^{12} and 2^{17}–2^{18}. In double precision the gain over `SimdFFT` is 1.3–2.0×, smaller because four doubles already fill half an AVX-512 register; `fastfft` beats FFTW up to 2^9 and at 2^{14} and 2^{19}, and FFTW is faster at 2^{10}–2^{12}, 2^{15}–2^{18} (down to 0.70×) and 2^{20}. Accuracy is unaffected: the largest error relative to $\max|X|$ was 2.1×10^{-7} (float) and 6.0×10^{-16} (double), against 4.8×10^{-7} and 8.4×10^{-16} for `SimdFFT` (Table 5). The confidence intervals are narrow (most within ±2 %), so these differences are not noise on this host.

Vector width. Compiling `fastfft` for narrower vectors separates the effect of width from that of the algorithm (Table 3). With 128 bit vectors, the width `SimdFFT` uses for floats, `fastfft` is 0.9–1.8× as fast as `SimdFFT`: the larger radices help mainly for small N. Going from 128 bit to 512 bit vectors then adds another factor of 1.4–2.8.

Emulated `simd`. Built with clang 20 and libc++, where vir-simd emulates the vector types, `fastfft` becomes 3.5–10.4× slower and `SimdFFT` 1.2–2.0× slower (Table 4). Under emulation `fastfft` is *slower* than `SimdFFT` for $N \geq 2^7$: its shuffles and broadcasts, which are single instructions with native types, become element loops. The patch below therefore enables `fastfft` only when `FASTFFT_NATIVE_SIMD` is set. gr4's default clang build uses libc++ and so falls into this case for both kernels.

In gr4. The patch (`gr4/0001-fastfft.patch`) adds the header to gr4 and makes `gr::algorithm::FFT` use it for complex power-of-two sizes ≥ 64; real-input and non-power-of-two transforms still use `SimdFFT`. In the spectrum block it also computes into the existing output buffer instead of a new vector per frame and fuses magnitude, phase and fftshift into one pass. gr4's FFT unit tests pass unchanged (GCC 14.2). Measured with gr4's own benchmark facility (Table 6, 5 alternating runs per build), the streaming block `FFT<T>` gets 1.7–2.5× faster for floats and 1.3–1.8× for doubles. The spectrum block `FFT<T,DataSet<P>>` (`bm_fft_block`, $N = 1024$) is unchanged within noise for complex float and real input (1.06×, 1.00×); the 1.55× for complex double we have not analysed. For complex float input it spends about 70 µs per frame, of which the FFT takes 0.7 µs; the rest goes to `hypot`, `atan2` and building the `DataSet`.

6 Discussion

Using the native vector width and fewer, larger radix passes is enough to make a portable FFT competitive with a hand-tuned library on the measured host; nothing in the kernel is specific to x86. FFTW's remaining lead at some sizes suggests that choosing radices per size, as FFTW's planner does, would help further. What `std::simd` lacks for this kernel is a two-input permute; with one (or with C++26 `std::simd` offering it), the kernel would be pure standard C++. The emulation result shows the other side of portability: code written for native vector types can become slower than simpler code when the types are emulated, so a library should check which backend it got.

These results come from one virtual AVX-512 core. AVX2-only x86, ARM with NEON or SVE, and other compilers may behave differently, and the harness was written so that collecting those data takes one command (`bench/run.sh`; `bench/gr4_compare.sh` for the gr4 patch). For gr4's spectrum block, the next gains lie outside the FFT.

Code. `https://github.com/dennisklein/fft` (header, harness, gr4 patch, raw data).

Contributions. D.K. posed the question, steered the work and reviewed the results. Claude, an AI model by Anthropic, wrote the code, ran the measurements and drafted the text.

References

[1] GNU Radio 4.0, `https://github.com/fair-acc/gnuradio4`, rev. `af62235`.

[2] J. Pommier, PFFFT: a pretty fast FFT, `https://bitbucket.org/jpommier/pffft`.

[3] ISO/IEC TS 19570:2018, Parallelism TS v2 (`std::experimental::simd`); P1928, `std::simd` for C++26.

[4] M. Kretz, vir-simd, `https://github.com/mattkretz/vir-simd`, v0.4.4.

[5] M. Frigo, S. G. Johnson, The design and implementation of FFTW3, Proc. IEEE 93(2):216–231, 2005.

[6] T. G. Stockham, High-speed convolution and correlation, AFIPS SJCC, 1966.

[7] C. Van Loan, Computational Frameworks for the Fast Fourier Transform, SIAM, 1992.

[8] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[9] T. Hoefler, R. Belli, Scientific benchmarking of parallel computing systems, SC'15, 2015.

Appendix: measurement data

Table 1: Complex float: median ns per transform [95 % CI] and speedup of fastfft (median of paired ratios [95 % CI]).

N	fastfft	gr4 SimdFFT	FFTW 3.3.10	vs. SimdFFT	vs. FFTW
2^6	35.0 [34.7, 35.2]	114 [112, 116]	44.1 [43.5, 44.6]	3.27 [3.23, 3.30]	1.26 [1.25, 1.27]
2^7	69.0 [68.5, 69.7]	237 [235, 238]	76.7 [76.3, 77.1]	3.42 [3.40, 3.44]	1.11 [1.10, 1.12]
2^8	162 [161, 164]	468 [464, 471]	162 [160, 163]	2.89 [2.87, 2.92]	0.99 [0.98, 1.00]
2^9	302 [298, 305]	1018 [1010, 1031]	356 [353, 362]	3.39 [3.36, 3.41]	1.16 [1.16, 1.18]
2^{10}	655 [649, 659]	2147 [2134, 2159]	759 [752, 764]	3.29 [3.25, 3.32]	1.17 [1.16, 1.17]
2^{11}	1793 [1784, 1807]	4942 [4917, 4959]	1696 [1682, 1708]	2.76 [2.75, 2.77]	0.95 [0.94, 0.96]
2^{12}	4895 [4866, 4922]	12.2 k [12.1 k, 12.2 k]	3854 [3805, 3892]	2.50 [2.48, 2.51]	0.79 [0.78, 0.79]
2^{13}	10.1 k [9997, 10.2 k]	29.3 k [29.2 k, 29.4 k]	12.6 k [12.5 k, 12.7 k]	2.91 [2.89, 2.93]	1.25 [1.24, 1.26]
2^{14}	20.5 k [20.3 k, 20.6 k]	59.1 k [58.8 k, 59.5 k]	28.1 k [27.9 k, 28.2 k]	2.90 [2.88, 2.93]	1.36 [1.34, 1.38]
2^{15}	45.6 k [45.2 k, 46.2 k]	134.7 k [134.2 k, 135.7 k]	62.0 k [61.5 k, 62.6 k]	2.96 [2.94, 2.98]	1.36 [1.34, 1.38]
2^{16}	140.9 k [140.4 k, 141.7 k]	271.7 k [269.7 k, 273.6 k]	139.1 k [138.4 k, 139.9 k]	1.93 [1.89, 1.97]	0.98 [0.97, 1.00]
2^{17}	418.3 k [410.7 k, 423.7 k]	726.4 k [719.1 k, 739.8 k]	370.4 k [365.2 k, 375.2 k]	1.77 [1.74, 1.79]	0.90 [0.88, 0.92]
2^{18}	1.008 M [1.001 M, 1.015 M]	1.808 M [1.792 M, 1.832 M]	881.0 k [873.5 k, 890.8 k]	1.80 [1.78, 1.81]	0.88 [0.87, 0.90]
2^{19}	2.068 M [2.059 M, 2.082 M]	4.355 M [4.319 M, 4.402 M]	4.238 M [4.187 M, 4.304 M]	2.11 [2.09, 2.13]	2.04 [2.01, 2.06]
2^{20}	5.524 M [5.439 M, 5.616 M]	9.040 M [8.948 M, 9.148 M]	8.338 M [8.246 M, 8.428 M]	1.64 [1.63, 1.66]	1.53 [1.51, 1.54]

Table 2: Complex double: median ns per transform [95 % CI] and speedup of fastfft (median of paired ratios [95 % CI]).

N	fastfft	gr4 SimdFFT	FFTW 3.3.10	vs. SimdFFT	vs. FFTW
2^6	72.8 [71.3, 75.2]	130 [128, 132]	84.5 [84.0, 84.9]	1.79 [1.78, 1.80]	1.13 [1.12, 1.17]
2^7	160 [159, 162]	286 [284, 288]	193 [191, 195]	1.80 [1.78, 1.82]	1.20 [1.19, 1.21]
2^8	383 [380, 386]	596 [589, 604]	396 [392, 400]	1.57 [1.55, 1.57]	1.03 [1.02, 1.03]
2^9	821 [811, 829]	1367 [1352, 1376]	866 [854, 890]	1.67 [1.65, 1.68]	1.05 [1.04, 1.09]
2^{10}	2155 [2146, 2166]	3218 [3202, 3236]	1857 [1824, 1944]	1.49 [1.48, 1.51]	0.85 [0.84, 0.88]
2^{11}	4952 [4927, 4985]	10.1 k [10.0 k, 10.2 k]	4406 [4314, 4482]	2.04 [2.02, 2.05]	0.87 [0.86, 0.89]
2^{12}	12.2 k [12.1 k, 12.3 k]	18.6 k [18.4 k, 18.7 k]	11.4 k [11.3 k, 11.5 k]	1.52 [1.51, 1.53]	0.93 [0.92, 0.95]
2^{13}	24.2 k [24.1 k, 24.3 k]	40.1 k [39.9 k, 40.3 k]	23.9 k [23.6 k, 24.3 k]	1.66 [1.65, 1.67]	1.00 [0.99, 1.00]
2^{14}	51.5 k [51.2 k, 51.8 k]	86.6 k [86.1 k, 87.3 k]	55.8 k [55.4 k, 56.2 k]	1.68 [1.67, 1.69]	1.09 [1.08, 1.10]
2^{15}	139.6 k [138.2 k, 140.2 k]	197.6 k [196.2 k, 199.4 k]	131.0 k [128.1 k, 133.0 k]	1.42 [1.41, 1.44]	0.95 [0.92, 0.97]
2^{16}	400.2 k [397.2 k, 407.1 k]	526.2 k [523.5 k, 532.6 k]	282.6 k [281.4 k, 284.5 k]	1.34 [1.32, 1.35]	0.70 [0.69, 0.71]
2^{17}	1.000 M [992.2 k, 1.008 M]	1.858 M [1.832 M, 1.871 M]	701.9 k [698.5 k, 713.0 k]	1.85 [1.83, 1.86]	0.71 [0.70, 0.71]
2^{18}	2.122 M [2.110 M, 2.149 M]	4.047 M [4.022 M, 4.090 M]	1.501 M [1.488 M, 1.511 M]	1.91 [1.89, 1.93]	0.71 [0.70, 0.71]
2^{19}	4.491 M [4.360 M, 4.675 M]	8.844 M [8.725 M, 8.946 M]	5.195 M [5.158 M, 5.242 M]	1.97 [1.94, 2.02]	1.21 [1.17, 1.22]
2^{20}	12.499 M [12.242 M, 12.947 M]	19.083 M [18.824 M, 19.385 M]	10.350 M [10.222 M, 10.491 M]	1.53 [1.50, 1.56]	0.85 [0.84, 0.87]

Table 3: Vector width, complex float: fastfft compiled for 128, 256, 512-bit vectors (`FASTFFT_SIMD_BYTES`), median ns per transform [95 % CI]; SimdFFT for reference.

N	128 bit	256 bit	512 bit	SimdFFT
2^6	64.4 [64.1, 66.4]	50.0 [49.5, 50.5]	35.0 [34.7, 35.2]	114 [112, 116]
2^7	135 [135, 136]	105 [104, 106]	69.0 [68.5, 69.7]	237 [235, 238]
2^8	333 [331, 335]	221 [220, 222]	162 [161, 164]	468 [464, 471]
2^9	690 [685, 696]	445 [442, 449]	302 [298, 305]	1018 [1010, 1031]
2^{10}	1502 [1492, 1513]	980 [977, 986]	655 [649, 659]	2147 [2134, 2159]
2^{11}	3323 [3313, 3340]	2332 [2300, 2375]	1793 [1784, 1807]	4942 [4917, 4959]
2^{12}	8781 [8742, 8819]	6660 [6515, 7907]	4895 [4866, 4922]	12.2 k [12.1 k, 12.2 k]
2^{13}	18.6 k [18.5 k, 18.7 k]	13.4 k [13.4 k, 13.6 k]	10.1 k [9997, 10.2 k]	29.3 k [29.2 k, 29.4 k]
2^{14}	40.6 k [39.3 k, 40.8 k]	28.2 k [27.9 k, 28.9 k]	20.5 k [20.3 k, 20.6 k]	59.1 k [58.8 k, 59.5 k]
2^{15}	129.7 k [129.2 k, 130.2 k]	73.8 k [73.5 k, 74.1 k]	45.6 k [45.2 k, 46.2 k]	134.7 k [134.2 k, 135.7 k]
2^{16}	296.3 k [294.1 k, 299.4 k]	201.6 k [200.1 k, 203.7 k]	140.9 k [140.4 k, 141.7 k]	271.7 k [269.7 k, 273.6 k]
2^{17}	642.0 k [639.1 k, 645.7 k]	458.7 k [454.1 k, 464.8 k]	418.3 k [410.7 k, 423.7 k]	726.4 k [719.1 k, 739.8 k]
2^{18}	1.432 M [1.423 M, 1.448 M]	1.083 M [1.072 M, 1.089 M]	1.008 M [1.001 M, 1.015 M]	1.808 M [1.792 M, 1.832 M]
2^{19}	3.753 M [3.724 M, 3.778 M]	2.538 M [2.520 M, 2.548 M]	2.068 M [2.059 M, 2.082 M]	4.355 M [4.319 M, 4.402 M]
2^{20}	8.402 M [8.341 M, 8.485 M]	5.956 M [5.845 M, 5.999 M]	5.524 M [5.439 M, 5.616 M]	9.040 M [8.948 M, 9.148 M]

Table 4: Standard library: complex float built with GCC/libstdc++ (native simd) and with clang/libc++ (vir-simd array emulation), median ns per transform [95 % CI].

N	fastfft, libstdc++	fastfft, libc++	SimdFFT, libstdc++	SimdFFT, libc++
2^6	35.0 [34.7, 35.2]	162 [160, 162]	114 [112, 116]	226 [225, 227]
2^7	69.0 [68.5, 69.7]	721 [716, 732]	237 [235, 238]	461 [457, 465]
2^8	162 [161, 164]	1382 [1375, 1392]	468 [464, 471]	879 [871, 887]
2^9	302 [298, 305]	2763 [2752, 2771]	1018 [1010, 1031]	1961 [1954, 1977]
2^{10}	655 [649, 659]	6141 [6097, 6186]	2147 [2134, 2159]	4039 [4019, 4097]
2^{11}	1793 [1784, 1807]	18.0 k [17.7 k, 18.4 k]	4942 [4917, 4959]	9468 [9323, 9630]
2^{12}	4895 [4866, 4922]	36.1 k [35.9 k, 36.2 k]	12.2 k [12.1 k, 12.2 k]	20.8 k [20.7 k, 20.9 k]
2^{13}	10.1 k [9997, 10.2 k]	73.0 k [72.5 k, 73.9 k]	29.3 k [29.2 k, 29.4 k]	46.8 k [46.6 k, 47.5 k]
2^{14}	20.5 k [20.3 k, 20.6 k]	158.3 k [157.5 k, 160.1 k]	59.1 k [58.8 k, 59.5 k]	97.8 k [96.2 k, 99.2 k]
2^{15}	45.6 k [45.2 k, 46.2 k]	403.3 k [399.4 k, 408.8 k]	134.7 k [134.2 k, 135.7 k]	211.8 k [210.6 k, 213.7 k]
2^{16}	140.9 k [140.4 k, 141.7 k]	839.8 k [831.3 k, 853.3 k]	271.7 k [269.7 k, 273.6 k]	444.1 k [441.4 k, 452.7 k]
2^{17}	418.3 k [410.7 k, 423.7 k]	1.738 M [1.724 M, 1.751 M]	726.4 k [719.1 k, 739.8 k]	1.046 M [1.036 M, 1.060 M]
2^{18}	1.008 M [1.001 M, 1.015 M]	3.664 M [3.646 M, 3.685 M]	1.808 M [1.792 M, 1.832 M]	2.322 M [2.307 M, 2.348 M]
2^{19}	2.068 M [2.059 M, 2.082 M]	8.803 M [8.709 M, 8.907 M]	4.355 M [4.319 M, 4.402 M]	5.370 M [5.314 M, 5.450 M]
2^{20}	5.524 M [5.439 M, 5.616 M]	19.276 M [19.058 M, 19.409 M]	9.040 M [8.948 M, 9.148 M]	11.230 M [11.106 M, 11.333 M]

Table 5: Accuracy: largest $\max_k |X_k - \hat{X}_k| / \max_k |\hat{X}_k|$ over all N, against a long-double reference.

	fastfft	SimdFFT	FFTW
float	2.1×10^{-7}	4.8×10^{-7}	2.4×10^{-7}
double	6.0×10^{-16}	8.4×10^{-16}	3.3×10^{-16}

Table 6: gnuradio4 blocks before and after the patch, gnuradio4's benchmark facility: median call time over runs, speedup as median of paired per-run ratios [95 % CI].

benchmark row	unpatched	patched	speedup
`std::complex<float> - fft`	70.5 µs	68.9 µs	1.06 [0.98, 1.07]
`std::complex<float> - fftCT`	79.7 µs	80.6 µs	1.00 [0.95, 1.01]
`std::complex<double> - fft`	93.0 µs	60.2 µs	1.55 [1.45, 1.59]
`std::complex<double> - fftCT`	98.0 µs	97.9 µs	0.99 [0.91, 1.21]
`float - fft`	23.3 µs	22.7 µs	1.00 [0.97, 1.07]
`double - fft`	27.4 µs	27.7 µs	1.01 [0.81, 1.10]
`FFT<float32> stream N= 64`	138.3 µs	53.5 µs	2.55 [2.53, 4.34]
`FFT<float32> stream N= 256`	135.3 µs	56.6 µs	2.39 [2.33, 4.00]
`FFT<float32> stream N= 1024`	150.4 µs	59.0 µs	2.52 [2.49, 2.57]
`FFT<float32> stream N= 4096`	210.3 µs	97.6 µs	2.14 [2.08, 2.18]
`FFT<float32> stream N= 16384`	256.6 µs	103.9 µs	2.48 [2.37, 2.60]
`FFT<float32> stream N= 65536`	300.1 µs	178.0 µs	1.70 [1.58, 1.73]
`FFT<float32> stream N= 262144`	1979.0 µs	1159.0 µs	1.76 [1.64, 1.83]
`FFT<float32> stream N=1048576`	9300.0 µs	5582.0 µs	1.66 [1.65, 1.73]
`FFT<float64> stream N= 64`	195.0 µs	144.3 µs	1.34 [1.19, 1.43]
`FFT<float64> stream N= 256`	228.3 µs	161.8 µs	1.42 [1.32, 1.47]
`FFT<float64> stream N= 1024`	274.7 µs	205.5 µs	1.34 [1.16, 1.37]
`FFT<float64> stream N= 4096`	367.3 µs	276.4 µs	1.33 [1.26, 1.41]
`FFT<float64> stream N= 16384`	444.5 µs	315.7 µs	1.43 [1.35, 1.60]
`FFT<float64> stream N= 65536`	616.6 µs	490.2 µs	1.33 [1.16, 1.55]
`FFT<float64> stream N= 262144`	4268.0 µs	2326.0 µs	1.84 [1.74, 1.87]
`FFT<float64> stream N=1048576`	19490.0 µs	11470.0 µs	1.74 [1.66, 1.76]